

Pruebas unitarias

JUnit · TDD · Tests parametrizados · Dobles de prueba

1. ¿Qué son las pruebas unitarias?

Una prueba unitaria verifica el correcto funcionamiento de un módulo individual del código —normalmente un método o una clase— de forma aislada, antes de comprobar cómo se integra con el resto del sistema.

Para que una prueba unitaria cumpla su función debe ser:

Característica	Qué significa en la práctica
Automatizable	Se ejecuta sin intervención manual, no «lo pruebo a mano cada vez»
Completa	Cubre la mayor cantidad posible de código, no solo el camino feliz
Reutilizable	No sirve solo para una ejecución puntual; se puede volver a correr cuando se quiera
Independiente	El resultado de una prueba no debe depender de que se haya ejecutado otra antes
Profesional	Se documenta y se cuida con la misma calidad que el código de producción

1. Pruebas unitarias — Ventajas

Las pruebas unitarias traen ventajas claras a medida que el proyecto crece:

- **Confianza al modificar**

Cualquier cambio que rompa algo se detecta enseguida.

- **Seguridad antes de integrar**

Se sabe que cada pieza funciona antes de combinarlas.

- **Documentación viva**

Los tests muestran cómo se espera que se use cada unidad de código.

- **Separación de interfaz e implementación**

Se puede cambiar cómo funciona algo por dentro sin tocar los tests si el comportamiento externo no cambia.

- **Localización de errores**

Se depura la unidad problemática en aislamiento, sin revisar todo el sistema a la vez.

2. JUnit

JUnit es el framework de pruebas unitarias más usado en Java. Permite comprobar que cada unidad de código se comporta como se espera y es una pieza habitual del desarrollo ágil. Se integra con Maven o Gradle para automatizar su ejecución dentro de la integración continua.

Conceptos clave

Concepto	Qué es
Prueba unitaria	Verifica una funcionalidad concreta de forma aislada
Assert	Instrucción que comprueba si el comportamiento obtenido coincide con el esperado
Anotación	Marca el papel que juega cada método dentro del ciclo de vida de la prueba
Clase de prueba	Agrupar los métodos de test relacionados con una misma clase de producción

2.1 Configuración básica con Maven

Para usar JUnit necesitas crear un proyecto Maven —una herramienta de construcción que gestiona la estructura de carpetas, la descarga de librerías externas y la compilación del proyecto—. Sin Maven tendrías que descargar los JARs a mano y configurar el classpath tú solo.

Estructura de carpetas que genera Maven automáticamente

```
mi-proyecto/  
├── pom.xml          <- configuración del proyecto  
└── src/  
    ├── main/java/   <- código de producción (tu aplicación)  
    └── test/java/    <- aquí irán tus clases de prueba
```

El archivo pom.xml (Project Object Model) es el corazón del proyecto Maven: describe el proyecto, su versión de Java y las dependencias —librerías externas que Maven descargará automáticamente desde Internet al construirlo—.

2.1 Maven — Dependencias

Una dependencia es una librería externa que el proyecto necesita. Sin Maven tendrías que descargar el JAR, guardarlo en alguna carpeta y añadirlo manualmente al classpath. Con Maven basta con declararlo en el pom.xml: Maven lo busca en Maven Central —el repositorio público de librerías Java— y lo descarga solo.

Dependencia de JUnit 5 — pegar dentro de <dependencies> en el pom.xml

```
<dependency>
  <groupId>org.junit.jupiter</groupId>      <!-- organización que la publica -->
  <artifactId>junit-jupiter</artifactId>     <!-- nombre del módulo concreto -->
  <version>5.11.4</version>                 <!-- versión que Maven descargará -->
  <scope>test</scope>                       <!-- solo para compilar y ejecutar tests -->
</dependency>
```

El <scope>test</scope> excluye la librería del JAR final de la aplicación. Para buscar cualquier dependencia: mvnrepository.com o search.maven.org. Tras pegar el XML, IntelliJ muestra un icono de Maven para descargarla.

2.2 Ciclo de vida de las anotaciones

Las anotaciones de JUnit controlan cuándo se ejecuta cada método. El orden es importante para preparar y limpiar el entorno de prueba correctamente:

Orden de ejecución

```
// --- inicio de la suite ---
@BeforeAll    // 1 vez

    @BeforeEach // antes de cada test
    @Test       // el test en sí
    @AfterEach  // después de cada test

// (se repite para cada @Test)

@AfterAll    // 1 vez al final
```

Anotación y cuándo se aplica

Anotación	Cuándo
@Test	Marca un método como prueba
@BeforeEach	Antes de cada prueba individual
@AfterEach	Después de cada prueba individual
@BeforeAll	Una sola vez, al inicio de la suite
@AfterAll	Una sola vez, al final de la suite

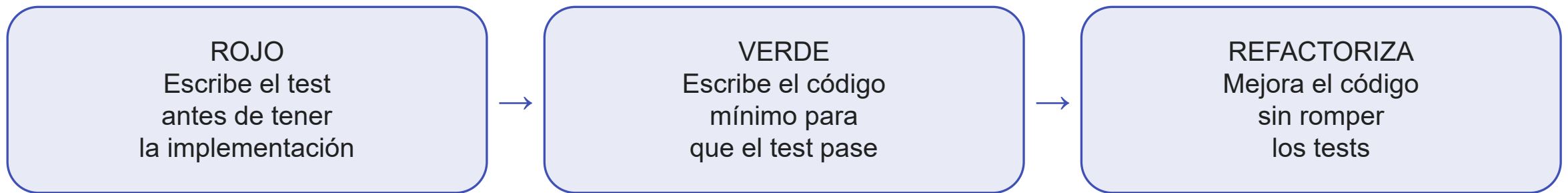
2.3 Los asserts principales

Un assert es la instrucción que verifica si el resultado obtenido coincide con el esperado. Si la condición no se cumple, JUnit marca el test como fallido y muestra qué se esperaba y qué se obtuvo.

Método	Comprueba que...
<code>assertEquals(esperado, obtenido)</code>	ambos valores son iguales
<code>assertTrue(condicion)</code>	la condición es verdadera
<code>assertFalse(condicion)</code>	la condición es falsa
<code>assertNull(objeto)</code>	el objeto es null
<code>assertNotNull(objeto)</code>	el objeto no es null
<code>assertThrows(Tipo.class, () -> { })</code>	el código del lambda lanza esa excepción concreta

2.4 TDD — Desarrollo guiado por pruebas

En el desarrollo normal escribes primero el código y luego los tests. TDD (Test-Driven Development) invierte ese orden: primero escribes un test que falla, luego el mínimo código para que pase, y finalmente mejoras el código sin romper los tests. El ciclo se repite para cada funcionalidad.



La ventaja de TDD no es solo tener tests: es que el diseño del código mejora porque lo piensas desde el punto de vista de quien lo va a usar, no de quien lo va a implementar. Si algo resulta difícil de testear, probablemente está mal diseñado.

El test debe fallar antes de escribir la implementación. Si el test pasa desde el principio, o está mal escrito, o la funcionalidad ya existía. Un test en verde sin haber implementado nada no aporta nada.

3. Tests parametrizados

Cuando se quiere probar el mismo método con varios conjuntos de datos distintos, escribir un `@Test` por cada combinación lleva a duplicar mucho código. Los tests parametrizados resuelven esto ejecutando un único método de prueba varias veces, una por cada conjunto de datos.

La anotación principal es `@ParameterizedTest`, que se combina con una fuente de datos:

Fuente	Cuándo usarla
<code>@ValueSource</code>	Una lista de valores de un mismo tipo (ints, strings...)
<code>@CsvSource</code>	Varias columnas de datos separadas por comas, directamente en el código
<code>@CsvFileSource</code>	Los datos se leen desde un archivo CSV externo
<code>@MethodSource</code>	Un método aparte genera los datos; útil cuando son más complejos

3. Tests parametrizados — Sin parametrizar vs. con @CsvSource

Ejemplo con Calculadora.sumar(): el mismo resultado con mucho menos código y sin duplicar la lógica de verificación.

Sin parametrizar — código duplicado

```
@Test
void testSumarValidos() {
    assertEquals(5, calc.sumar(2, 3));
    assertEquals(7, calc.sumar(3, 4));
}

@Test
void testSumarNegativo() {
    assertThrows(
        IllegalArgumentException.class,
        () -> calc.sumar(-1, 3));
}
```

Con @CsvSource — un solo test

```
@ParameterizedTest
@CsvSource({
    "2, 3, 5",
    "3, 4, 7"
})
void testSumarValidos(int a,
    int b, int esperado) {
    assertEquals(esperado,
        calc.sumar(a, b));
}
```

3. Tests parametrizados — @ValueSource

Cuando solo se necesita un parámetro por caso, @ValueSource es más directo: se pasa una lista de valores del mismo tipo y JUnit ejecuta el test una vez por cada elemento de la lista.

Ejemplo: validar que varios números son pares

```
// JUnit ejecuta este test 4 veces: con 2, con 4, con 6 y con 8
@ParameterizedTest
@ValueSource(ints = {2, 4, 6, 8})
void testEsParConNumerosValidos(int numero) {
    assertTrue(validador.esPar(numero));
}

// JUnit ejecuta este test 3 veces: con -2, con -4 y con -6
@ParameterizedTest
@ValueSource(ints = {-2, -4, -6})
void testEsParConNumerosNegativos(int numero) {
    assertThrows(IllegalArgumentException.class,
        () -> validador.esPar(numero));
}
```

4. Dobles de prueba

A veces probar una clase implica, sin querer, probar también sus dependencias: una base de datos, un servicio web, un archivo externo... Eso complica la prueba y mezcla responsabilidades. Los dobles de prueba son objetos que sustituyen esas dependencias reales dentro del test.

Tipos de dobles

Tipo	Qué hace
Mock	Objeto falso que simula una dependencia real. Se define qué devuelve cada método y se puede verificar si fue llamado y con qué parámetros.
Stub	Versión simplificada que se limita a devolver datos predefinidos, sin la lógica de verificación de un mock.
Dummy	Objeto sin funcionalidad real; solo existe porque el código lo necesita para ejecutarse (ej: parámetro obligatorio que no influye en el resultado).

4.1 Mocks con Mockito

Mockito es la librería más usada en Java para crear mocks. Permite definir qué devuelve cada método de una dependencia falsa y verificar que fue llamado correctamente durante el test.

Añadir Mockito como dependencia en el pom.xml

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>5.15.2</version>
  <scope>test</scope>
</dependency>
```

El problema sin mock: si CalculadoraService usa Calculadora internamente y Calculadora tiene un error, el test de CalculadoraService fallaría aunque el servicio estuviera bien escrito. Con un mock controlamos exactamente qué devuelve Calculadora, y el test solo depende de la lógica del propio servicio.

4.1 Mocks con Mockito — Ejemplo completo

CalculadoraService.java

```
public class CalculadoraService {
    private Calculadora calc;

    public CalculadoraService(
        Calculadora calc) {
        this.calc = calc;
    }

    public int dobleDeLaSuma(
        int a, int b) {
        int suma = calc.sumar(a, b);
        return calc.multiplicar(suma, 2);
    }
}
```

CalculadoraServiceTest.java

```
@Test
void testDobleDeLaSuma() {
    // 1. Crear el mock
    Calculadora mock =
        mock(Calculadora.class);

    // 2. Configurar qué devuelve
    when(mock.sumar(2, 3))
        .thenReturn(5);
    when(mock.multiplicar(5, 2))
        .thenReturn(10);

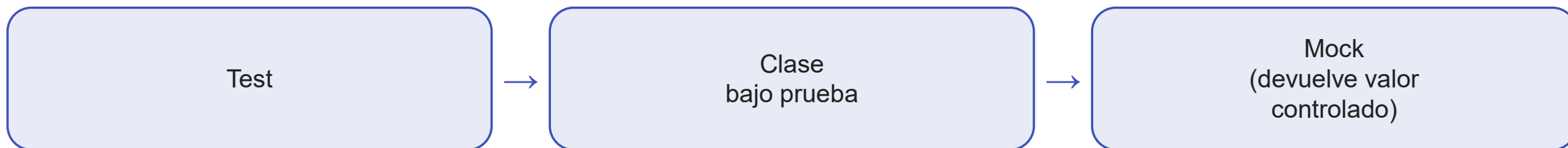
    // 3. Instanciar con el mock
    CalculadoraService svc =
        new CalculadoraService(mock);

    // 4. Ejecutar y verificar
    assertEquals(10,
        svc.dobleDeLaSuma(2, 3));
}
```

4.1 Mocks con Mockito — ¿Por qué aislar?

Cuando un test usa un mock en lugar de la dependencia real, el resultado solo depende de la lógica de la clase bajo prueba. Si el test falla, el error está en esa clase, no en la dependencia. Sin mock, un error en Calculadora haría fallar también el test de CalculadoraService, aunque el servicio esté bien escrito.

El cocinero y sus herramientas: imagina que quieres evaluar a un cocinero (CalculadoraService) que usa ciertas herramientas (Calculadora). Si quieres juzgar solo al cocinero, no tiene sentido usar herramientas defectuosas de verdad: le das herramientas simuladas que sabes que funcionan perfectamente. Eso es exactamente lo que hace un mock dentro de una prueba unitaria.



El test controla exactamente lo que devuelve el mock, de modo que el resultado solo depende de la lógica de la clase que se está evaluando.